

ACTIVIDAD EVALUADA 1

Desarrollo de Interfaz Frontend

1. Descripción de la actividad

Los estudiantes deberán desarrollar una solución frontend que dé respuesta a una problemática mediante el diseño y construcción de una interfaz web funcional. El proyecto deberá utilizar **HTML5, CSS3, Bootstrap y JavaScript**, incorporando **Git, GitHub y Git Flow** para el control de versiones y trabajo colaborativo.

El resultado final será expuesto frente al curso. Durante la presentación, el docente podrá realizar preguntas técnicas individuales para comprobar el dominio de las tecnologías y del código desarrollado.

2. Tecnologías

- Git y comandos básicos.
- GitHub.
- Git Flow.
- HTML5.
- CSS3.
- Bootstrap.
- JavaScript.

3. Problemáticas propuestas

1. Sistema de reservas de espacios universitarios

Una universidad cuenta con salas de clases, laboratorios, salas de estudio y espacios de reunión que son utilizados por estudiantes, docentes y organizaciones estudiantiles. Actualmente, la disponibilidad de estos espacios es difícil de consultar y los usuarios no cuentan con una interfaz centralizada para realizar sus solicitudes.

El equipo deberá diseñar una interfaz que permita **consultar la disponibilidad de espacios y gestionar solicitudes de reserva**.

La solución debería considerar, al menos:

- Visualización de los espacios disponibles.
- Información relevante de cada espacio: nombre, ubicación, capacidad y características.
- Búsqueda y filtros por edificio, capacidad o tipo de espacio.
- Selección de fecha y horario.
- Formulario para solicitar una reserva.
- Visualización de las reservas realizadas.
- Posibilidad de cancelar una reserva.
- Mensajes visuales para confirmar acciones o informar errores.
- Diseño adaptable para computadores, tablets y teléfonos.

Los datos podrán ser simulados mediante JavaScript.

2. Plataforma de gestión de actividades académicas

Los estudiantes deben organizar actividades provenientes de distintas asignaturas: tareas, controles, proyectos, presentaciones y evaluaciones. A medida que avanza el semestre, puede resultar difícil mantener un control de las actividades pendientes y sus respectivas fechas de entrega.

El equipo deberá desarrollar una interfaz que permita **organizar y visualizar las actividades académicas de un estudiante**.

La solución debería considerar, al menos:

- Visualización de las asignaturas.
- Listado de actividades académicas.
- Nombre y descripción de cada actividad.
- Fecha de entrega.
- Asignatura asociada.
- Estado de la actividad: pendiente, en progreso o completada.
- Filtros por asignatura y estado.
- Formulario para registrar nuevas actividades.
- Posibilidad de modificar o eliminar actividades.
- Indicadores que permitan visualizar el avance académico.
- Identificación visual de actividades próximas a vencer.

Los datos podrán ser simulados y administrados mediante estructuras de datos en JavaScript.

3. Sistema de atención y gestión de turnos

Una organización recibe diariamente una gran cantidad de personas que requieren atención. Para ordenar el proceso, se desea implementar un sistema de turnos que permita a los usuarios obtener un número y visualizar el estado de la atención.

El equipo deberá desarrollar una interfaz que **simule el funcionamiento de un sistema de atención mediante turnos**.

La solución debería considerar, al menos:

- Interfaz para obtener un nuevo número de atención.
- Visualización del número que está siendo atendido.
- Identificación del módulo o ventanilla que está atendiendo.
- Visualización de las personas que se encuentran esperando.
- Diferenciación entre distintos tipos de atención.
- Botón o mecanismo para llamar al siguiente turno.
- Eliminación del turno una vez atendido.
- Indicadores sobre personas atendidas y personas en espera.
- Mensajes visuales para informar el estado del sistema.

El sistema no requiere comunicación con dispositivos físicos ni backend. Toda la simulación deberá realizarse mediante HTML, CSS, Bootstrap y JavaScript.

4. Marketplace universitario

Dentro de una comunidad universitaria existe un mercado informal donde los estudiantes compran y venden libros, computadores, accesorios, instrumentos, artículos deportivos y otros productos. Actualmente, estas publicaciones se encuentran dispersas en diferentes canales y resulta difícil buscar productos específicos.

El equipo deberá diseñar una interfaz de **marketplace orientado exclusivamente a la comunidad universitaria**.

La solución debería considerar, al menos:

- Catálogo de productos publicados.
- Imagen, nombre, precio y descripción del producto.
- Categorías de productos.
- Buscador.
- Filtros por categoría y rango de precio.
- Vista detallada de un producto.
- Opción para agregar productos a favoritos.
- Carrito de compra.
- Formulario para publicar un nuevo producto.
- Posibilidad de eliminar publicaciones.
- Información básica del vendedor.

No es necesario implementar pagos, autenticación real ni comunicación entre usuarios. Las operaciones deberán ser simuladas utilizando JavaScript.

5. Panel de gestión de un emprendimiento

Un pequeño emprendimiento necesita una herramienta visual que permita a su administrador conocer el estado general del negocio. Actualmente, la información sobre ventas, productos y pedidos se encuentra dispersa, por lo que se requiere una interfaz que permita obtener una visión rápida de la operación.

El equipo deberá desarrollar un **dashboard frontend para la gestión de un pequeño negocio**.

La solución debería considerar, al menos:

- Resumen general del negocio.
- Cantidad de ventas realizadas.
- Monto total vendido.
- Productos disponibles.
- Productos con bajo stock.
- Listado de pedidos.
- Estado de cada pedido.
- Información básica de clientes.
- Filtros para consultar información.
- Tablas, tarjetas o indicadores visuales.
- Representaciones gráficas simples cuando sean pertinentes.
- Navegación entre las distintas secciones del sistema.

Toda la información podrá ser simulada mediante arrays y objetos JavaScript. **No se requiere implementar una base de datos ni un backend.**

4. Restricciones técnicas

Obligatorio:

- HTML5 semántico.
- Bootstrap para apoyar la construcción de la interfaz.
- CSS3 para personalización visual y estilos propios.
- JavaScript para lógica e interacción.
- Manipulación del DOM.
- Manejo de eventos.
- Formularios con validación.
- Información dinámica mediante JavaScript.
- Git, GitHub y Git Flow.
- Diseño responsive.

No permitido:

- React, Angular o Vue.
- Otros frameworks frontend.
- Backend.
- Bases de datos.
- APIs externas.

Bootstrap debe utilizarse como herramienta de apoyo. No basta con usar componentes prediseñados: se evaluará la capacidad para combinar Bootstrap con HTML semántico, CSS propio y JavaScript.

5. Estructura mínima del proyecto

```
proyecto/  
├── index.html  
├── css/  
│   └── styles.css  
├── js/  
│   └── app.js  
├── assets/  
├── ...  
└── README.md
```

6. Git, GitHub y Git Flow

- Commits frecuentes y descriptivos.
- Ramas main y develop.
- Al menos una rama feature/.
- Integración mediante Pull Request.
- README con integrantes, problemática, tecnologías e instrucciones.
- Responsabilidades respaldadas por el historial del repositorio.

7. Requisitos mínimos funcionales

- Resolver la problemática seleccionada.
- Interacción real del usuario.
- JavaScript modificando dinámicamente la interfaz.
- Al menos un formulario con validación.
- Adaptación a dispositivos móviles.
- Datos simulados mediante arrays y objetos JavaScript.
- Uso pertinente de componentes/utilidades Bootstrap.
- Personalización mediante CSS propio.

8. Presentación y defensa técnica — 20%

Tiempo de exposición máximo 10 minutos.

Criterio	Ponderación
Presentación de la problemática y solución	4%

Demostración funcional	6%
Explicación de decisiones técnicas	5%
Capacidad de responder preguntas técnicas	5%
TOTAL	20%

9. Rúbrica técnica — 80%

Criterio técnico	Ponderación	Aspectos a evaluar
HTML5 y estructura semántica	12%	Semántica, jerarquía, formularios, atributos y organización.
Bootstrap y construcción de interfaz	13%	Sistema de grillas, componentes, utilidades, responsive y combinación con HTML/CSS.
CSS y personalización visual / responsive	10%	CSS propio, selectores, box model, Flexbox/Grid, tipografía, espaciado e identidad visual.
JavaScript y lógica de programación	20%	Variables, condicionales, ciclos, funciones, arrays, objetos, eventos y organización.
Interactividad y manipulación del DOM	15%	Agregar, eliminar y modificar elementos; filtros; estados; renderizado y actualización.
Formularios y validaciones	5%	Inputs, campos obligatorios, validaciones y mensajes de error.
Git, GitHub y Git Flow	5%	Commits, ramas, Git Flow, Pull Requests, colaboración y documentación.
TOTAL TÉCNICO	80%	

10. Defensa técnica individual

El docente podrá realizar preguntas técnicas individuales y aleatorias. Las preguntas podrán abordar HTML, CSS, Bootstrap, JavaScript, Git/GitHub y decisiones de implementación.

Ejemplos:

- ¿Por qué utilizaron esta etiqueta HTML en lugar de un div?
- ¿Cómo funciona el sistema de grillas de Bootstrap utilizado?
- ¿Qué diferencia hay entre container y container-fluid?
- ¿Qué componentes o utilidades de Bootstrap utilizaron y por qué?
- ¿Qué parte del diseño realizaron con CSS propio?
- ¿Cómo validan este formulario?
- ¿Qué evento JavaScript ejecuta esta función?
- ¿Cómo están modificando el DOM?
- ¿Qué diferencia existe entre let y const?
- ¿Qué diferencia existe entre main y develop?
- ¿Qué funcionalidad implementaron en esta rama feature?

11. Resultado esperado

El producto final deberá corresponder a una pequeña aplicación frontend funcional que permita resolver de manera clara y coherente la problemática seleccionada. Para su desarrollo, se deberá utilizar **HTML5** para estructurar el contenido, **Bootstrap** como apoyo en la construcción de la interfaz, **CSS3** para la personalización visual y el diseño responsive, y **JavaScript** para implementar la lógica, interactividad y comportamiento dinámico de la aplicación.

El proyecto deberá evidenciar, además, un flujo de trabajo colaborativo y ordenado mediante el uso de **Git, GitHub y Git Flow**, reflejado en el historial de commits, ramas y Pull Requests del repositorio.

Uso de herramientas de Inteligencia Artificial

Para esta actividad está **estrictamente prohibido utilizar herramientas de generación automática de código o agentes de desarrollo**, tales como GitHub Copilot, Cursor, Claude Code, ChatGPT u otras herramientas equivalentes, para producir total o parcialmente el código que será evaluado.

El objetivo de esta restricción es evaluar directamente las competencias de los estudiantes en **HTML, CSS, Bootstrap y JavaScript**, así como su capacidad para implementar y comprender las soluciones desarrolladas.

Se permite utilizar herramientas de Inteligencia Artificial **exclusivamente como apoyo para resolver dudas conceptuales, comprender errores, consultar documentación o reforzar contenidos**, siempre que el código evaluado sea escrito y comprendido por los propios estudiantes.

Durante la presentación, el docente podrá realizar preguntas técnicas sobre cualquier parte del código desarrollado. Si un estudiante no es capaz de explicar adecuadamente una implementación que forma parte de su proyecto, esta situación podrá ser considerada en la evaluación de la defensa técnica.